

A02_CVE-2022-23650

Here is a snippet of code written in go. I want to check for any potential security vulnerabilities.

package logic

```
import (  
    "errors"  
    "fmt"  
    "time"  
  
    "github.com/golang-jwt/jwt/v4"  
    "github.com/gravitl/netmaker/logger"  
    "github.com/gravitl/netmaker/models"  
    "github.com/gravitl/netmaker/servercfg"  
)  
  
var jwtSecretKey []byte  
  
// SetJWTSecret - sets the jwt secret on server startup  
func SetJWTSecret() {  
    currentSecret, jwtErr := FetchJWTSecret()  
    if jwtErr != nil {  
        jwtSecretKey = []byte(RandomString(64)) // 512 bit  
        random password  
        if err := StoreJWTSecret(string(jwtSecretKey)); err != nil {  
            logger.FatalLog("something went wrong when  
configuring JWT authentication")  
        }  
    } else {  
        jwtSecretKey = []byte(currentSecret)  
    }  
}  
  
// CreateJWT func will used to create the JWT while signing  
in and signing out  
func CreateJWT(uuid string, macAddress string, network  
string) (response string, err error) {  
    expirationTime := time.Now().Add(5 * time.Minute)  
    claims := &models.Claims{  
        ID:      uuid,
```

```

        Network: network,
        MacAddress: macAddress,
        StandardClaims: jwt.StandardClaims{
            Issuer: "Netmaker",
            Subject: fmt.Sprintf("node| %s", uuid),
            IssuedAt: time.Now().Unix(),
            ExpiresAt: expirationTime.Unix(),
        },
    },
}

token := jwt.NewWithClaims(jwt.SigningMethodHS256,
claims)
tokenString, err := token.SignedString(jwtSecretKey)
if err == nil {
    return tokenString, nil
}
return "", err
}

// CreateUserJWT - creates a user jwt token
func CreateUserJWT(username string, networks []string,
isAdmin bool) (response string, err error) {
    expirationTime := time.Now().Add(60 * 12 * time.Minute)
    claims := &models.UserClaims{
        UserName: username,
        Networks: networks,
        IsAdmin: isAdmin,
        StandardClaims: jwt.StandardClaims{
            Issuer: "Netmaker",
            IssuedAt: time.Now().Unix(),
            Subject: fmt.Sprintf("user| %s", username),
            ExpiresAt: expirationTime.Unix(),
        },
    },
}

token := jwt.NewWithClaims(jwt.SigningMethodHS256,
claims)
tokenString, err := token.SignedString(jwtSecretKey)
if err == nil {
    return tokenString, nil
}
return "", err
}

```

```

}

// VerifyToken func will used to Verify the JWT Token while
using APIS
func VerifyUserToken(tokenString string) (username string,
networks []string, isadmin bool, err error) {
    claims := &models.UserClaims{}

    if tokenString == servercfg.GetMasterKey() {
        return "masteradministrator", nil, true, nil
    }

    token, err := jwt.ParseWithClaims(tokenString, claims,
func(token *jwt.Token) (interface{}, error) {
    return jwtSecretKey, nil
})

    if token != nil && token.Valid {
        // check that user exists
        if user, err := GetUser(claims.UserName);
user.UserName != "" && err == nil {
            return claims.UserName, claims.Networks,
claims.IsAdmin, nil
        }
        err = errors.New("user does not exist")
    }
    return "", nil, false, err
}

// VerifyToken - gRPC [nodes] Only
func VerifyToken(tokenString string) (nodeID string, mac
string, network string, err error) {
    claims := &models.Claims{}

    //this may be a stupid way of serving up a master key
    //TODO: look into a different method. Encryption?
    if tokenString == servercfg.GetMasterKey() {
        return "mastermac", "", "", nil
    }

    token, err := jwt.ParseWithClaims(tokenString, claims,
func(token *jwt.Token) (interface{}, error) {

```

```
        return jwtSecretKey, nil
    })

    if token != nil {
        return claims.ID, claims.MacAddress, claims.Network,
        nil
    }
    return "", "", "", err
}
```

Below is a security review of the provided Go code, which involves JWT creation, verification, and secret key management. Here are the potential vulnerabilities and recommendations to address them:

Potential Vulnerabilities and Issues:

1. Hardcoded or Insufficient Secret Management:

- **Issue:** The `jwtSecretKey` is set once at startup using `SetJWTSecret()` with a random string of 64 characters if no secret is fetched.
- **Concern:**
 - If the secret is regenerated on every startup, previously issued JWTs will become invalid.
 - Relying on `RandomString` without robust entropy might introduce vulnerabilities.

Recommendation:

- Store the secret securely (e.g., in an environment variable or encrypted secrets manager).
 - Use a consistent secret to avoid invalidating tokens across restarts.
-

2. Insecure Master Key Logic:

- **Issue:** The token string is compared with a master key (`servercfg.GetMasterKey()`).
 - If the master key is known, it can be used to bypass authentication and impersonate any user or node.

Recommendation:

- Avoid using master keys. If absolutely necessary, ensure that:
 - It is protected and never hardcoded.
 - It is encrypted or stored securely.
-

3. Use of `HS256` for JWT Signing:

- **Issue:** The code uses `HS256` (HMAC-SHA256) to sign JWTs. Although it is secure if implemented correctly, there are risks if the key is leaked or reused inappropriately.
 - Additionally, symmetric signing (HS256) shares the same key between signer and verifier, which can be exploited if compromised.

Recommendation:

- Consider using asymmetric algorithms such as `RS256` . This will allow only the server to sign the JWT, and other parties to verify it using the public key.
-

4. Token Expiration and Refreshing:

- **Issue:** The tokens are issued with expiration times (5 minutes for nodes and 12 hours for users), but there's no mechanism for refreshing expired tokens. This could lead to usability issues or unnecessary re-authentication.

Recommendation:

- Implement a **refresh token** system to avoid frequent logouts.
 - Use short expiration for access tokens and allow refresh tokens to renew them.
-

5. Lack of Token Blacklisting:

- **Issue:** No mechanism is provided to invalidate or revoke JWTs (e.g., on password change, user deactivation, etc.). A compromised token could remain valid until it expires.

Recommendation:

- Implement token blacklisting, possibly using a distributed in-memory store (e.g., Redis) to track revoked tokens.
-

6. Weak Error Handling in JWT Parsing:

- **Issue:** The `VerifyToken` and `VerifyUserToken` functions rely on the presence of `token.Valid` to determine the validity of the JWT. However, no explicit handling of token parsing errors is done.

Recommendation:

- Ensure proper validation and error handling:

```
go

if err != nil || !token.Valid {
    return "", "", "", errors.New("invalid token")
}
```

7. Exposure of Error Messages:

- **Issue:** In some cases, the code returns raw error messages (e.g., `"user does not exist"`). This could leak information to attackers.

Recommendation:

- Avoid leaking internal errors. Use generic messages like `"invalid credentials"` and log the detailed error for internal monitoring.
-

8. Inconsistent Error Checks for `GetUser` :

- **Issue:** The code checks if `user.UserName` is empty to determine if the user exists but doesn't properly handle errors returned by `GetUser` .

Recommendation:

- Validate both the user object and the error:

```
go

if err != nil || user.UserName == "" {
    return "", nil, false, errors.New("user not found")
}
```

Summary of Recommended Changes:

1. Secret Management:

- Use environment variables or a secrets manager for `jwtSecretKey` .
- Ensure consistency of secrets between restarts.

2. Master Key Handling:

- Avoid using master keys or secure them appropriately.

3. JWT Signing Algorithm:

- Consider switching to `RS256` for stronger security.

4. Token Lifecycle Management:

- Implement refresh tokens.
- Add token blacklisting support.

5. Error Handling:

- Improve error handling during token parsing and validation.
 - Avoid exposing internal error messages.
-

With these improvements, the code will be more secure against potential attacks, such as token forgery, brute-force attacks on secrets, and misuse of master keys.

Provide the exact part of the code along with line numbers that are vulnerable or have security issues.

Here are the specific vulnerabilities with exact lines in the provided code, along with suggestions for fixes:

1. Secret Management Issues

Code Location:

```
go

var jwtSecretKey []byte // Line 12
```

```
go

func SetJWTSecret() { // Line 15
    currentSecret, jwtErr := FetchJWTSecret() // Line 16
    if jwtErr != nil {
        jwtSecretKey = []byte(RandomString(64)) // Line 18
        if err := StoreJWTSecret(string(jwtSecretKey)); err != nil { // Line 19
            logger.FatalLog("something went wrong when configuring JWT
authentication") // Line 20
        }
    } else {
        jwtSecretKey = []byte(currentSecret) // Line 23
    }
}
```

- **Issue:** If `FetchJWTSecret` fails, a new secret is generated each time the server starts. This invalidates previously issued tokens.
- **Fix:**
 - Use a consistent secret, ideally stored in environment variables or a secure vault.

- Add entropy validation for `RandomString` generation.
-

2. Weak JWT Signing Algorithm

Code Location:

```
go

token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims) // Lines 37, 63
tokenString, err := token.SignedString(jwtSecretKey) // Lines 38, 64
```

- **Issue:** `HS256` is used, which is symmetric. If the secret key is leaked, anyone can generate valid tokens.
 - **Fix:** Switch to an asymmetric algorithm such as `RS256`.
-

3. Insecure Master Key Logic

Code Location:

```
go

if tokenString == servercfg.GetMasterKey() { // Lines 78, 103
    return "masteradministrator", nil, true, nil // Line 79
}
```

```
go

if tokenString == servercfg.GetMasterKey() { // Line 103
    return "mastermac", "", "", nil // Line 104
}
```

- **Issue:** Using a hardcoded or overly privileged master key can lead to privilege escalation if leaked.

- **Fix:** Avoid master keys altogether. If unavoidable, store them securely and restrict their usage.
-

4. Missing Refresh Token Mechanism and Short Expiration

Code Location:

```
go

expirationTime := time.Now().Add(5 * time.Minute) // Line 31
expirationTime := time.Now().Add(60 * 12 * time.Minute) // Line 57
```

- **Issue:** Access tokens are short-lived, but there is no refresh token mechanism to extend the session securely.
 - **Fix:** Implement a refresh token mechanism to reduce frequent logouts.
-

5. Weak Error Handling During Token Verification

Code Location:

```
go

token, err := jwt.ParseWithClaims(tokenString, claims, func(token *jwt.Token)
(interface{}, error) { // Lines 84, 109
    return jwtSecretKey, nil // Lines 85, 110
}))
if token != nil && token.Valid { // Line 87
    // check that user exists
    if user, err := GetUser(claims.UserName); user.UserName != "" && err == nil { //
Line 89
        return claims.UserName, claims.Networks, claims.IsAdmin, nil // Line 90
    }
    err = errors.New("user does not exist") // Line 92
}
```

- **Issue:** The error from `jwt.ParseWithClaims` is not handled properly.
- **Fix:** Add proper error handling for token parsing.

```
go

if err != nil || !token.Valid {
    return "", nil, false, errors.New("invalid token")
}
```

6. Exposure of Internal Errors

Code Location:

```
go

err = errors.New("user does not exist") // Line 92
```

- **Issue:** Exposing internal error messages can leak information to attackers.
- **Fix:** Use generic error messages.

```
go

err = errors.New("invalid credentials")
```

7. Inconsistent User Existence Check

Code Location:

```
go

if user, err := GetUser(claims.UserName); user.UserName != "" && err == nil { //
Line 89
```

- **Issue:** The code checks if the username is empty but does not properly validate the error from `getUser`.
- **Fix:**

```
go

if err != nil || user.UserName == "" {
    return "", nil, false, errors.New("user not found")
}
```

Summary of Vulnerabilities by Line Number

Line Range	Issue	Recommendation
12	Unprotected secret key	Store in environment variable or vault
15-23	Inconsistent secret key on restart	Use persistent or pre-configured secret
37, 63	Use of <code>HS256</code>	Switch to <code>RS256</code> for stronger security
78-79, 103-104	Insecure master key logic	Avoid or secure master key usage
31, 57	No refresh token mechanism	Implement refresh tokens
84-85, 109-110	Missing error handling for JWT parsing	Handle <code>err</code> and <code>token.Valid</code> properly
92	Leaking internal error messages	Use generic error messages
89	Inconsistent user existence check	Validate both <code>user</code> and <code>err</code>

These adjustments will improve the security of the code, preventing token forgery, privilege escalation, and other potential vulnerabilities.

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Here is a detailed security analysis of your Go code with vulnerabilities mapped to the **OWASP Top 10 Web Application Security Risks**:

1. Insecure JWT Secret Management

Code Location:

```
go

var jwtSecretKey []byte // Line 12

jwtSecretKey = []byte(RandomString(64)) // Line 18
```

Vulnerability:

- If `FetchJWTSecret()` fails, a new random secret is generated every time the server starts, invalidating previously issued JWTs.
- This inconsistency can impact the system's reliability and result in tokens becoming invalid unexpectedly.

OWASP Mapping:

- **[A2:2021 - Cryptographic Failures]**
 - Secret keys should be securely managed, stored, and consistently used across system restarts.
 - The use of a weak or improperly managed secret could lead to token forgery if the key generation or retrieval is compromised.

Fix:

- Store the secret key securely (e.g., in an environment variable or a vault) to maintain consistency across restarts.

2. Insecure JWT Signing Algorithm (HS256)

Code Location:

```
go
```

```
token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims) // Lines 37, 63
```

Vulnerability:

- Using a symmetric algorithm (HS256) means both the signing and verification processes rely on the same secret key. If this key is leaked, anyone can generate valid tokens.

OWASP Mapping:

- [A2:2021 - Cryptographic Failures]
 - This weakness can allow attackers to impersonate valid users if the secret key is compromised.

Fix:

- Use an asymmetric signing algorithm like RS256 with a private key for signing and a public key for verification.
-

3. Insecure Master Key Usage

Code Location:

```
go

if tokenString == servercfg.GetMasterKey() { // Lines 78, 103
    return "masteradministrator", nil, true, nil // Line 79
}
```

Vulnerability:

- A hardcoded or statically stored master key can bypass the authentication mechanism. If this key is leaked or guessed, attackers can impersonate privileged users.

OWASP Mapping:

- [A5:2021 - Security Misconfiguration]
 - Using master keys without adequate protection can introduce significant security risks.

Fix:

- Avoid using master keys, or store them securely (e.g., in a secrets manager) with strict access control.
-

4. Missing Refresh Token Mechanism

Code Location:

```
go

expirationTime := time.Now().Add(5 * time.Minute) // Line 31
expirationTime := time.Now().Add(60 * 12 * time.Minute) // Line 57
```

Vulnerability:

- Tokens have a short expiration time, but there is no refresh token mechanism. This could lead to user inconvenience and frequent re-authentication.
- Attackers may exploit expired tokens by trying to reuse them.

OWASP Mapping:

- **[A7:2021 - Identification and Authentication Failures]**
 - Proper session management is crucial. Without refresh tokens, users are forced to re-login frequently, making the system more susceptible to token-related attacks (e.g., token replay).

Fix:

- Implement a refresh token mechanism to extend sessions securely without forcing users to re-login frequently.
-

5. Improper Error Handling and Information Disclosure

Code Location:

```
go
```

```
err = errors.New("user does not exist") // Line 92
```

Vulnerability:

- Revealing internal system details (e.g., "user does not exist") allows attackers to gather information about the system's internals and existence of users.

OWASP Mapping:

- [A9:2021 - Security Logging and Monitoring Failures]
 - Error messages should not leak sensitive information to users. Instead, use generic error messages while logging detailed errors for internal use.

Fix:

- Replace with:

```
go

err = errors.New("invalid credentials")
```

6. Weak Error Handling during JWT Verification

Code Location:

```
go

token, err := jwt.ParseWithClaims(tokenString, claims, func(token *jwt.Token)
(interface{}), error) { // Lines 84, 109
    return jwtSecretKey, nil // Lines 85, 110
})
if token != nil && token.Valid { // Line 87
    if user, err := GetUser(claims.UserName); user.UserName != "" && err == nil { //
Line 89
        return claims.UserName, claims.Networks, claims.IsAdmin, nil // Line 90
    }
    err = errors.New("user does not exist") // Line 92
}
```

Vulnerability:

- Error handling for JWT parsing is incomplete. If parsing fails, the code should return immediately rather than relying on further logic.

OWASP Mapping:

- **[A7:2021 - Identification and Authentication Failures]**
 - Weak validation logic can allow attackers to bypass security measures.

Fix:

- Ensure proper error handling:

```
go

if err != nil || !token.Valid {
    return "", nil, false, errors.New("invalid token")
}
```

7. Lack of Token Blacklisting Mechanism

Vulnerability:

- JWTs remain valid until they expire, with no mechanism for revocation (e.g., if a user logs out or changes their password).

OWASP Mapping:

- **[A7:2021 - Identification and Authentication Failures]**
 - Without token revocation, compromised tokens remain valid until they expire.

Fix:

- Implement token blacklisting using an in-memory store (e.g., Redis) to invalidate tokens when necessary.

Summary of Vulnerabilities and OWASP Mapping

Vulnerability	Code Location	OWASP Top 10 Mapping	Recommendation
Insecure JWT Secret Management	Lines 12, 15-23	A2:2021 - Cryptographic Failures	Use environment variables or vaults for secure key storage.
Insecure JWT Signing Algorithm (HS256)	Lines 37, 63	A2:2021 - Cryptographic Failures	Switch to RS256 for stronger security.
Insecure Master Key Usage	Lines 78-79, 103-104	A5:2021 - Security Misconfiguration	Avoid master keys or store them securely with strict access.
Missing Refresh Token Mechanism	Lines 31, 57	A7:2021 - Identification and Authentication Failures	Implement a refresh token mechanism.
Improper Error Handling and Information Disclosure	Line 92	A9:2021 - Security Logging and Monitoring Failures	Use generic error messages and log detailed ones internally.
Weak Error Handling during JWT Verification	Lines 84-87, 109-110	A7:2021 - Identification and Authentication Failures	Ensure robust error handling during token parsing.
Lack of Token Blacklisting Mechanism	JWT-related functions	A7:2021 - Identification and Authentication Failures	Implement token blacklisting to invalidate tokens as needed.

Conclusion

The code presents several common security vulnerabilities, including **insecure cryptographic practices**, **weak authentication logic**, **misconfiguration**, and **inadequate error handling**. Addressing these issues by following the OWASP guidelines will significantly enhance the security and reliability of your Go application.

For each identified vulnerability, provide an exploit example that an attacker could use. Include a step-by-step explanation of how the exploit works, the necessary conditions or setup, any potential limitations or obstacles

an attacker might face, and the likelihood of this attack happening. Ensure the exploits are specific to the vulnerability type provided.

1. Insecure JWT Secret Management

Exploit Example: Generate forged JWTs due to predictable or inconsistent secret key management.

Steps to Exploit:

1. If `FetchJWTSecret()` fails during server startup, the code generates a new random secret using `RandomString(64)`.
2. An attacker with access to the server could trigger frequent restarts, ensuring a new secret is generated each time.
3. The attacker could replay a previously valid token because the secret has changed, bypassing verification.
4. Alternatively, if the random string generation function (`RandomString`) is weak or predictable, an attacker could guess the secret and forge valid JWTs.

Necessary Conditions:

- Server restarts frequently without persisting the `jwtSecretKey`.
- `RandomString` implementation lacks sufficient entropy or uses a predictable algorithm.

Limitations:

- Attacker needs access to force server restarts or observe the secret generation process.

Likelihood:

- **Moderate:** If `RandomString` is secure, this is harder to exploit. However, unpredictable secrets that change frequently can still cause operational issues.

2. Insecure JWT Signing Algorithm (HS256)

Exploit Example: Forging tokens due to symmetric key exposure.

Steps to Exploit:

1. The application uses `HS256` for signing, meaning the same key is used for both signing and verifying JWTs.
2. An attacker gains access to the `jwtSecretKey` (e.g., via a leaked environment variable or through exploiting other vulnerabilities).
3. Using the key, the attacker crafts a valid token with elevated privileges, e.g., setting `isAdmin: true`.
4. The attacker sends the token to the server, and it passes validation, granting unauthorized access.

Necessary Conditions:

- The `jwtSecretKey` is leaked or obtained by the attacker.

Limitations:

- Requires the attacker to first compromise the secret key.

Likelihood:

- **High:** Shared secrets are a common target for attackers, especially in applications with weak secret management practices.
-

3. Insecure Master Key Usage

Exploit Example: Bypass authentication using the master key.

Steps to Exploit:

1. The code allows bypassing JWT verification if the provided token matches the master key (`servercfg.GetMasterKey()`).
2. An attacker guesses or obtains the master key (e.g., via brute force, logging vulnerabilities, or weak key storage).
3. The attacker sends the master key as the token.
4. The server grants unrestricted access, treating the attacker as an administrator.

Necessary Conditions:

- Master key is hardcoded, weakly stored, or guessable.

- No rate-limiting or monitoring for repeated login attempts.

Limitations:

- Requires knowledge or guessing of the master key.
- Strong rate-limiting can make brute-forcing the key infeasible.

Likelihood:

- **Moderate:** Depends on the secrecy and strength of the master key.
-

4. Missing Refresh Token Mechanism

Exploit Example: Session expiration attack due to user frustration.

Steps to Exploit:

1. A token expires after 5 minutes (nodes) or 12 hours (users). There is no mechanism for refreshing tokens.
2. An attacker intercepts an expired token (e.g., via a man-in-the-middle attack or session logs).
3. The attacker uses the expired token in a replay attack.
4. The server, without a blacklist or refresh token mechanism, may process the expired token if its validation logic is weak.

Necessary Conditions:

- Lack of refresh token and token revocation mechanisms.
- The server misvalidates expired tokens.

Limitations:

- Attack depends on how the server handles expired tokens.
- HTTPS could mitigate token interception.

Likelihood:

- **Low to Moderate:** Replay attacks are rare but can be critical if the system misvalidates expired tokens.

5. Improper Error Handling and Information Disclosure

Exploit Example: Enumerating valid users by analyzing error messages.

Steps to Exploit:

1. The code returns specific error messages such as `"user does not exist"`.
2. An attacker sends authentication requests with random usernames.
3. When the error `"user does not exist"` is returned, the attacker knows the username is invalid.
4. By identifying valid usernames, the attacker can launch brute force or phishing attacks.

Necessary Conditions:

- The system returns detailed error messages.

Limitations:

- Rate-limiting or CAPTCHA can slow down the enumeration process.

Likelihood:

- **High:** Attackers frequently exploit detailed error messages for enumeration.

6. Weak Error Handling During JWT Verification

Exploit Example: Invalid token bypass due to poor validation logic.

Steps to Exploit:

1. The server parses a JWT token with invalid claims or an invalid signature.
2. If `jwt.ParseWithClaims` returns `token != nil`, but `err != nil` and `!token.Valid`, the server incorrectly processes the request due to incomplete error handling.
3. The attacker crafts a malformed token to bypass certain security checks or crash the application.

Necessary Conditions:

- Poorly handled token validation logic.

- Server ignores `err` or `!token.Valid`.

Limitations:

- Exploiting logic flaws can require significant trial and error.

Likelihood:

- **Moderate:** Developers often overlook robust error handling during token validation.
-

7. Lack of Token Blacklisting Mechanism

Exploit Example: Reuse of revoked tokens.

Steps to Exploit:

1. A token is issued to a legitimate user but later revoked (e.g., due to password change or account deactivation).
2. Without a token blacklist, the token remains valid until expiration.
3. An attacker intercepts or uses a previously issued token to access the application even after revocation.

Necessary Conditions:

- No token blacklist implemented.
- Token revocation scenarios not handled.

Limitations:

- Requires prior access to a legitimate token.
- Short token expiration times can mitigate the issue.

Likelihood:

- **Moderate:** Common in systems without token revocation mechanisms.
-

Likelihood and Risk Summary

Vulnerability	Likelihood	Impact	Overall Risk
Insecure JWT Secret Management	Moderate	High	Moderate
Insecure JWT Signing Algorithm	High	High	High
Insecure Master Key Usage	Moderate	High	Moderate
Missing Refresh Token Mechanism	Low to Moderate	Moderate	Moderate
Improper Error Handling	High	Moderate	Moderate to High
Weak Error Handling in JWT Parsing	Moderate	Moderate	Moderate
Lack of Token Blacklisting Mechanism	Moderate	High	Moderate

Recommendations to Mitigate Exploits

1. **Securely store and manage JWT secret keys** using environment variables or a secrets vault.
2. **Switch to RS256** or an asymmetric algorithm for JWT signing.
3. **Eliminate master keys** or implement strict access controls and monitoring for their use.
4. **Add refresh tokens and token revocation mechanisms** for better session management.
5. **Replace specific error messages with generic ones** and log detailed errors for internal diagnostics.
6. **Ensure robust token validation** by properly handling errors and verifying token validity.
7. **Implement a token blacklist** to revoke compromised or invalid tokens in real time.